

Evaluating ML-KEM at (much) higher security levels

Swechchha Adhikari¹, Vadim Lyubashevsky², and Peter Schwabe^{3,4}

¹ Brigham Young University, Provo, USA
mail@swechchhaa.com

² IBM Research, Zurich, Switzerland
VAD@zurich.ibm.com

³ Max Planck Institute for Security and Privacy, Bochum, Germany
peter@cryptojedi.org

⁴ Radboud University, Nijmegen, The Netherlands

1 Introduction

After a multi-year, multi-round public competition, NIST published its first post-quantum standards FIPS-203 [25], FIPS-204 [24], and FIPS-205 [26] in 2024, specifying ML-KEM, ML-DSA, and SLH-DSA, respectively. In 2025, NIST selected an additional KEM, HQC, for standardization in the fourth round. Unlike ML-KEM, which is based on module-LWE, HQC relies on the Quasi-Cyclic Syndrome Decoding (QCSD) problem, providing diversity beyond lattice-based assumptions [18]. This choice also marked a shift away from the lattice-based KEMs that dominated the Round 3 submissions. Four of the schemes in the PQC Round 3 Submissions [23] — Kyber [10] (standardized as ML-KEM), Saber [13], NTRU LPrime [8], and FrodoKEM [9] — are closely related lattice KEMs, spanning a spectrum of algebraic structure, from FrodoKEM’s unstructured LWE, through Kyber’s and Saber’s cyclotomic modules, to the prime-degree number fields of NTRU Prime. This spectrum reflects different design principles, commonly understood as structure vs. conservatism. ML-KEM’s module structure gives small keys and fast NTT arithmetic, but is also seen as a potential attack surface, so conservative designs discard it entirely [9]. FrodoKEM, for example, is based on a more conservative hardness assumption of plain LWE, which is why bodies such as BSI, ENISA, and NLNCSA point to it for long-term confidentiality [29]; it was also recently standardized by ISO [20].

It is useful to separate the two notions of conservatism this framing bundles together as they concern two different (and today, entirely hypothetical) classes of attack. The first arises from the choice of hardness assumption. FrodoKEM rests on unstructured LWE to avoid a hypothetical structure break, an attack that would exploit the algebraic structure of ring/module LWE but not apply directly to plain LWE. The second involves the security margin a scheme carries above its target level, which protects against advances in the best known attacks today. This could include improved lattice reduction, dual attacks, or sharper cost models that would affect the concrete security of every lattice scheme at its current parameters, including FrodoKEM. Both of these hedges are distinct,

but not entirely independent: additional parameter margin can also increase the cost of a hypothetical structural attack, while removing structure can affect the cost of known attacks. A third, orthogonal hedge is to change the hardness assumption altogether, as HQC does by relying on QCSD instead of LWE. This provides cryptographic diversity, since an attack on one assumption does not automatically imply an attack on the other. However, diversity alone also does not imply complete cryptanalytic independence. Recent work has demonstrated connections between lattice and code-based cryptanalysis, including adaptations of lattice-sieving techniques to codes [14]. Moreover, the quasi-cyclic structure of HQC has been noted to have important parallels with the ring and module structures used in structured lattice schemes [27]. These approaches address different sources of uncertainty, and it is impossible to predict which is more likely to matter.

Faced with this uncertainty, increasing the margin has been a historically standard response, provided that sufficient bandwidth is available to accommodate the increase. For computationally secure schemes, a concrete security estimate is the cost of the best attack known at the time, and such estimates evolve as cryptanalysis matures. For instance, the recommended size of an RSA modulus climbed from 512 to 1024 to 2048 bits and beyond as factoring algorithms and available computation advanced [7]. Lattice estimates are not static either. The cost models used for estimation and the attacks have evolved over more than a decade [12, 2]. The core framework for estimating the security of LWE-based schemes has nevertheless remained stable, with BKZ-based lattice reduction at its core. Within this framework, the cost of the best known attacks continues to drift in both directions as improvements are proposed [17, 21], scrutinized [15], and refined [11].

If the best known attacks continue to improve, how much margin is required remains open, and the useful question is not whether today’s KEMs carry enough but how much margin they can carry at all. This reframing matters because additional margin comes at a cost, requiring larger parameters and hence higher bandwidth and runtime, so no scheme can indefinitely withstand advances in cryptanalysis simply by scaling its parameters. For LWE-family KEMs there is a further constraint, since the parameters that determine security also control the decryption failure probability, so increasing the noise to gain a larger security margin necessarily affects correctness.

Crucially, what structure determines is the *price* of buying additional margin. At matched security levels, an unstructured scheme already requires roughly an order of magnitude more bandwidth than a module scheme [9, 10]. Since these communication costs are considered practical enough for conservative deployments, a module scheme can spend the same bandwidth budget on larger parameters and obtain a conservative security margin while retaining the efficiency advantages of NTT arithmetic.

Recent work has explored this high-margin regime from the perspective of unstructured LWE. KyFrog [22] increases the LWE dimension and adjusts the noise distribution to achieve substantially higher security estimates for much

larger ciphertexts. This motivates the complementary question of how much security margin a module KEM can achieve within the communication budget of a conservative unstructured LWE scheme, and at what computational cost. We answer this question with KAIBURR, a module KEM that scales ML-KEM’s module rank and introduces a generalized noise distribution to preserve correctness. We first characterize the correctness and security limits of naively increasing ML-KEM’s module rank, then introduce a noise family that allows further scaling. We then build three different parameter sets with the new noise sampler, and implement them across conventional software and dedicated hardware-accelerated platforms. Finally, we compare KAIBURR with existing KEMs, including FrodoKEM and HQC. More specifically, our contributions can be summarized as follows.

1.1 Our contributions

- *We show that naively scaling ML-KEM’s module rank hits a correctness wall.* Raising k alone, with Kyber’s noise and modulus fixed, satisfies the 2^{-128} decryption-failure bound only up to $k = 7$ and fails already at $k = 8$, so larger module rank cannot buy additional security margin on its own (Section 3).
- *We introduce a one-parameter noise family that allows us to scale further.* Our family $f(t)$ generalizes Kyber’s centered binomial distribution, thinning its ± 2 tail geometrically as t grows while leaving the ± 1 mass and its branchless sampling unchanged. This gives a noise knob that can be traded against module rank to keep decryption failure below 2^{-128} as k increases (Section 4).
- *We select concrete parameter sets that reach large security margins within FrodoKEM-640’s communication budget.* Using this trade-off, we obtain three parameter sets, kaiburr-1792, -4608, and -6144, at $k = 7, 18, 24$, whose classical/quantum Core-SVP security climbs from 479/434 to 1717/1557 bits while their public keys and ciphertexts stay within the communication budget of FrodoKEM-640, its smallest standardized parameter set (see Section 6).
- *We build KAIBURR across three implementation targets.* We implement all three parameter sets in C, in Jasmin with machine-checked constant-time verification, and as an ACC variant targeting the Pavona hardware root of trust, detailing the engineering consequences of scaling k and removing ciphertext compression (Section 5).
- *We benchmark KAIBURR against FrodoKEM and HQC.* Across three Intel microarchitectures, our AVX2 and Jasmin implementations run substantially faster than FrodoKEM-640 and achieve performance comparable to HQC, despite having (much) larger security margins. (Section 6).

1.2 Access to artifacts

All software, including parameter analysis scripts, implementation source code, and other artifacts, is publicly available on [GitHub](#).

2 Preliminaries

3 Warmup: Just changing k

The main mechanism to increase bit security in Kyber is to raise the module rank k [4]. Fixing the ML-KEM base ring, noise distribution, and modulus, and varying only k , we measure the decryption-failure probability using the scripts provided by KYBER team.⁵ Throughout we use the centered binomial noise CBD_2 (the noise of ML-KEM-768 and -1024; recall that ML-KEM-512 uses CBD_3). The ranks $k = 3$ and 4 recover the standardized sets ML-KEM-768, and -1024, while $k \geq 5$ is an extrapolation. Table 1 reports the results. Against the 2^{-128} failure bound expected at this security level, the bound holds at $k = 7$ ($2^{-133.7}$) but fails at $k = 8$ ($2^{-116.8}$), so raising k further cannot buy higher security without sacrificing correctness.

These values are computed without ciphertext compression, and therefore are optimistic estimates. The rounding applied to $\mathbf{u}$ and v in Kyber introduces additional error, which can only increase the true failure probability. The actual failure probabilities are expected to be higher than those reported here, and the correctness limit on k may be even more restrictive.

Table 1. Decryption-failure probability and public-key and ciphertext sizes as a function of module rank k , at fixed ML-KEM noise and modulus, without ciphertext compression. Ranks $k = 3, 4$ correspond to ML-KEM-768, and -1024.

k	Failure probability	pk (bytes)	ct (bytes)
3	$2^{-299.6}$	1184	1536
4	$2^{-231.1}$	1568	1920
5	$2^{-186.5}$	1952	2304
6	$2^{-155.9}$	2336	2688
7	$2^{-133.7}$	2720	3072
8	$2^{-116.8}$	3104	3456

4 Revisiting the noise distribution

Even though the resulting public key and ciphertext sizes at $k = 7$ remain relatively small, increasing k further is not possible without affecting correctness. Instead, one could reduce the failure probability at large k by increasing the modulus q . We note that all variants of KYBER fix $q = 3329$, which is explicitly justified in the submission documentation to NIST. The modulus $q = 3329$ is the small prime satisfying the NTT-enabling condition $n \mid (q - 1)$, notably weaker than the $2n \mid (q - 1)$ one might expect, which KYBER deliberately does not

⁵ <https://github.com/pq-crystals/security-estimates>

require [5]. It was reduced from the round-1 value 7681 to recover the bandwidth spent by dropping public-key compression and co-tuned to balance ciphertext size, failure probability, and security.

Since ML-KEM’s standardization, highly optimized and high-assurance implementations have emerged across software and hardware platforms [28,6,1]. Changing q would mean not being able to reuse any existing optimized routines in software or hardware, which sacrifices an important engineering advantage and results in a different scheme altogether.

This is not to say that k is the only parameter that can be modified. We focus on changes that preserve the *essence* of Kyber: the ring $\mathbb{Z}_q[X]/(X^{256} + 1)$, the NTT implementation and its constants, and the existing analysis framework. The module rank k qualifies trivially. So does the error distribution. Since the noise only affects the error term and does not interact with the arithmetic structure or the NTT, it can be replaced without altering the core implementation. What it changes are only the MLWE hardness estimate and the decryption-failure probability, both of which can be recomputed through the same scripts, over the same ring.

In this sense, the error distribution is another degree of freedom that moves both security and correctness without changing what the scheme is at its core. This is the direction we pursue and where our main contribution lies.

To scale past $k = 7$, we need a noise knob that pushes the failure-driving part of the error back down as k grows, while preserving KYBER’s *essence*. Equally important, it should retain the cheap, branchless, constant-time sampling that motivates KYBER’s use of a centered binomial in the first place. This requirement rules out simply using a discrete Gaussian, as in FrodoKEM.

4.1 A parameterized noise distribution

We introduce a one-parameter family $f(t)$ on $\{-2, -1, 0, 1, 2\}$, the same support as CBD_2 , defined by

$$\Pr[\pm 2] = 2^{-t}, \quad \Pr[\pm 1] = \frac{1}{4}, \quad \Pr[0] = \frac{1}{2} - 2^{-(t-1)}.$$

The error at ± 1 is fixed. The single parameter t acts only on the ± 2 tail, thinning it geometrically. A sample is drawn from uniform bits $a_1, \dots, a_t$ as

$$x = (2a_t - 1) \left(1 - a_1 + 2 \prod_{i=1}^{t-1} a_i \right),$$

which is branchless and therefore samples in constant time; we machine-check both constant-time (CT) and speculative constant-time (SCT) in Jasmin [3] (see Section 5). The standard deviation $\sigma(t) = \sqrt{0.5 + 2^{3-t}}$ decreases from 1 at $t = 4$ toward $\sqrt{0.5}$ as t grows.

Crucially, $f(t)$ generalizes Kyber’s own noise distribution. At $t = 4$, the probabilities become

$$\Pr[\pm 2] = \frac{1}{16}, \quad \Pr[\pm 1] = \frac{1}{4}, \quad \Pr[0] = \frac{3}{8},$$

exactly matching CBD_2 , the noise distribution used by ML-KEM-768 and -1024. As $t \rightarrow \infty$, the ± 2 mass vanishes and $f(t)$ approaches CBD_1 .

4.2 Why thinning the tail is the efficient trade

A key property of $f(t)$ is that increasing t reduces the probability of the ± 2 terms exponentially while leaving the ± 1 mass unchanged. As defined above, we have

$$\Pr_{f(t)}[\pm 2] = 2^{-t}, \quad \Pr_{f(t)}[\pm 1] = \frac{1}{4}.$$

So, t is the tunable parameter for thinning the tail of the distribution without changing the bulk ± 1 terms. This gives finer control over the noise than the standard centered binomial choices. In particular, CBD_1 and CBD_2 have standard deviations $\sqrt{0.5}$ and 1, respectively, with no centered binomial parameter providing an intermediate distribution. As shown above, $f(t)$ ranges from CBD_2 toward CBD_1 as t increases, giving us finer control over the noise.

This finer control is useful as the module rank k increases. Increasing k increases the amount of error accumulated during encryption and therefore makes the decryption-failure constraint more restrictive. Rather than moving directly from CBD_2 to CBD_1 , we can increase t as k grows, reducing the ± 2 contribution while keeping the ± 1 contribution fixed.

4.3 Selecting t

Since t is the number of uniform bits per sample, vectorization-friendly values are natural, especially considering the AVX2 implementations. As shown above, $t = 4$ recovers the CBD_2 sampler, so it serves as our anchor. Notice that $t = 8$ consumes exactly one PRF byte per coefficient, meaning it admits one-byte-per-coefficient SIMD packing without cross-byte bit-splitting. This makes $t = 8$ a very convenient choice. Our third choice, $t = 6$, sits in the middle and does not share the byte-alignment of $t = 4$ or $t = 8$. Sampling four coefficients consumes $6 \times 4 = 24$ bits, i.e. three bytes of PRF output, which is not as natural to implement as the byte-aligned $t = 4$ and $t = 8$ cases. It is nonetheless interesting relative to $t = 8$, for two reasons. First, it retains more of the ± 2 tail, giving a higher standard deviation and more security per dimension. Second, it spends fewer PRF bytes per coefficient (0.75 vs. 1), which is relevant on platforms *without* Keccak acceleration. For each choice of t , we pick the highest k satisfying the failure probability bound of 2^{-128} , staying within the communication budget of FrodoKEM640.

We use FrodoKEM-640 as our baseline for parameter selection and comparison throughout the paper because it is the smallest standardized FrodoKEM parameter set and therefore imposes the tightest communication budget of 9,616 public-key bytes and 9,700 ciphertext bytes. This gives us a restrictive baseline for asking how much security margin KAIBURR can achieve within the communication cost of a conservative lattice-based KEM.

The three parameter sets we carry forward are $f(4)$, $f(6)$, and $f(8)$, at $k = 7$, 18, and 24 respectively, which follow from these choices (Table 2).

Table 2. KAIBURR parameter sets. Classical/quantum security (Core-SVP) and failure probability computed with the Kyber team’s scripts; other parameters as in ML-KEM, compression removed.

Scheme	k	noise	failure	class.	quant.	pk (B)	ct (B)
kaiburr-1792	7	$f(4)$	$2^{-133.7}$	479	434	2720	3072
kaiburr-4608	18	$f(6)$	$2^{-134.1}$	1281	1162	6944	7296
kaiburr-6144	24	$f(8)$	$2^{-139.9}$	1717	1557	9248	9600

4.4 The ceiling of the family

The family $f(t)$ is not unbounded. The lightest noise it can produce is its limit $f(\infty)$, with $\Pr[\pm 1] = \frac{1}{4}$, $\Pr[0] = \frac{1}{2}$, $\Pr[\pm 2] = 0$, and minimal variance $\sigma^2 = 0.5$. At $n = 256$ and $q = 3329$, the failure probability crosses 2^{-128} at $k = 30$, so $k = 29$ is the largest rank that $f(t)$ can support (Table 3). Reaching higher security levels would require another change, such as modifying q , n , or the noise family itself, moving beyond the KYBER structure we seek to preserve.

Table 3. Failure probability under the limiting distribution $f(\infty)$ (minimum variance $\sigma^2 = 0.5$) as k increases, uncompressed. The 2^{-128} bound is first violated at $k = 30$, so $k = 29$ is the ceiling of the $f(t)$ family at $n = 256$, $q = 3329$.

k	Failure probability	pk (bytes)	ct (bytes)
24	$2^{-158.7}$	9248	9600
26	$2^{-146.2}$	10016	10368
28	$2^{-135.5}$	10784	11136
29	$2^{-130.6}$	11168	11520
30	$2^{-126.1}$	11552	11904

5 Implementation

We provide three implementations of KAIBURR, each forked from an existing ML-KEM implementation: a C implementation, a Jasmin implementation with machine-checked constant-time verification, and an assembly implementation for Pavona’s Asymmetric Cryptography Coprocessor (ACC). The ACC reuses ML-KEM’s arithmetic core and accelerates Keccak in hardware. This provides us with a complementary hardware-accelerated setting where we can evaluate the cost of scaling the module rank k . As upstream, the C and Jasmin code each

have a portable `ref` path and an AVX2-vectorized path; ACC has a single hand-written path.

All three implementations make the same three changes to ML-KEM:

1. the module rank k is scaled to $\{7, 18, 24\}$, giving the parameter sets kaiburr-1792, -4608, and -6144 (named kaiburr4, kaiburr6, and kaiburr8 in the code, after their noise parameter t ; see Table 2);
2. ciphertext compression is removed, so both ciphertext polynomials are serialized at full 12-bit precision; and
3. the centered binomial sampler is replaced by a sampler for the noise family $f(t)$, with $t \in \{4, 6, 8\}$.

Everything else, including the ring, the modulus $q = 3329$, the NTT and its constants, the Fujisaki–Okamoto transform, and the symmetric primitives, is left unchanged.

These changes nonetheless affect more code than they touch directly. ML-KEM implementations assume $k \leq 4$ in several places, and the AVX2 packers assume a particular coefficient order. Most of the rewritten code follows from scaling k , which we describe first (Section 5.1). We then cover the noise sampler (Section 5.2), the removal of compression (Section 5.3), and what is checked about each implementation (Section 5.4).

5.1 Scaling k

Accumulator overflow. Larger values of k can overflow the matrix-vector product. In the NTT domain, each polynomial of the product is a sum of k pointwise products. ML-KEM accumulates this sum in an `int16_t` and reduces it once at the end. Each Montgomery product has magnitude below q , so the sum fits whenever $kq < 2^{15}$, i.e. $k \leq 9$. This holds for the standardized $k \leq 4$, but fails for our large parameters $k = 18$ or $k = 24$.

All three implementations therefore reduce the accumulator after every pointwise product. We apply this in all cases, including for $k = 7$, where it is not strictly necessary. The cost is one extra reduction per term i.e. a Barrett reduction in the C and Jasmin code, and a modular addition against the `MOD` register on ACC.

Matrix generation. The AVX2 matrix sampler in ML-KEM hardcodes how the k^2 matrix entries are grouped into batches of four for the 4-way Keccak for each $k \leq 4$. We replace this with a generic loop; it samples four entries at a time with 4-way Keccak and handles any remaining entries with a scalar tail. Only $k = 7$ has a remainder, of one entry.

The same loop generates both $\mathbf{A}$ for key generation and its transpose $\mathbf{A}^T$ for encapsulation, by selecting the matching domain separators. The C code computes these separators from the row and column indices, while the Jasmin code reads them from a precomputed table. ACC generates one entry at a time and derives each separator from a running nonce, with separate handling for the transposed and non-transposed orders.

Memory. The C and Jasmin code store the full matrix $\mathbf{A}$ on the stack. Its size grows quadratically with k , from 24.5 KiB at $k = 7$ to 288 KiB at $k = 24$. This is not suitable for ACC, whose stock configuration only provides 32 KiB of data memory. That is less than the working set of a single decapsulation at $k = 24$, which is about 34 KiB for the ciphertext, re-encrypted ciphertext, secret key, and hashing buffers alone, before the matrix is counted.

We address this in two ways. First, the ACC implementation never stores $\mathbf{A}$. It generates one polynomial at a time, multiplies it into a running accumulator, and discards it. Second, we enlarge ACC’s visible data memory from 31 KiB to 127 KiB in the hardware description, out of 128 KiB of physical DMEM including a 1 KiB scratch region. The ACC results in Section 6 are therefore for this modified hardware configuration.

5.2 The noise sampler

The noise family $f(t)$ maps t uniformly random bits $a_1, \dots, a_t$, read least-significant first, to the coefficient

$$(2a_t - 1) \left(1 - a_1 + 2 \prod_{i < t} a_i \right).$$

All our implementations replace the centered binomial sampler with this expression, and the PRF output grows to $t \cdot n/8$ bytes per polynomial. The five variants differ in how they compute two parts of the expression: the product term $\prod_{i < t} a_i$ and the sign $(2a_t - 1)$. Table 4 summarizes their choices.

Table 4. How each implementation computes the product term and applies the sign in $f(t)$.

Variant	Product term	Sign	Word size
C ref	bit-by-bit AND chain	integer multiply	scalar
C AVX2	compare against a mask	conditional lane negate	8–32-bit lanes
Jasmin ref	carry propagation	conditional two’s-complement	scalar
Jasmin AVX2	compare against a mask	conditional two’s-complement	8–16-bit lanes
ACC	carry propagation	modular subtract + select	16-bit lanes

The Jasmin reference code and the ACC assembly avoid the literal AND chain with a carry trick. Adding one to the low $t - 1$ bits carries into bit $t - 1$ exactly when all of those bits are set, so a right shift gives us the product. The AVX2 kernels instead isolate the same bits and apply a vector compare, which gives the product directly as an all-ones or all-zero mask. In every vectorized and assembly variant, the sign is then applied separately and without branching.

Extracting t -bit fields from the PRF output is simplest for $t = 8$ and $t = 4$, which align with bytes and nibbles, and hardest for $t = 6$. Both AVX2 kernels

use an additional shuffle or gather to align the 6-bit fields. The C kernel reads past the end of its input, so the PRF buffer carries 32 bytes of trailing slack. ACC has no byte-shuffle instruction, so it instead chains `bn.rshi` shifts to move a 6-bit window across register boundaries.

Key generation needs $2k$ noise polynomials. The AVX2 code samples them in groups of four using the 4-way Keccak.

5.3 Removing compression

KAIBURR serializes both ciphertext polynomials $\mathbf{u}$ and v at full 12-bit precision, so the ciphertext grows to $(k + 1) \cdot 384$ bytes. The public key keeps its size of $384k + 32$ bytes, since ML-KEM never compresses it.

The reference code and ACC reuse their existing 12-bit packer unchanged. The AVX2 code needs a new one. Its existing packer is used only for the public key, which is stored in the permuted coefficient order of the AVX2 NTT, so it applies that permutation while packing. Ciphertext polynomials leave the inverse NTT in standard order. Both AVX2 forks therefore add a packer and unpacker that work in standard order, without the permutation.

The new packer emits 24 bytes per iteration as two 16-byte stores twelve bytes apart, so its final iteration would write past the end of the buffer. The C code stages the final iteration through a scratch buffer, and the Jasmin code handles it as a separate case.

5.4 Assurance

Jasmin. The Jasmin implementation is a fork of `formosa-mlkem` [6], whose EasyCrypt proofs establish functional correctness for ML-KEM. These proofs do not carry over to KAIBURR, since the sampler, module rank, and serialization all change. What we check is much narrower.

Every exported function carries two annotations. A `ct` annotation is checked by Jasmin’s constant-time checker, for both standard and speculative constant-time. A `safety` annotation establishes that array accesses stay in bounds and that all memory is initialized before use. Both properties are checked for all six builds.

Testing. The C implementation relies on the deterministic round-trip self-tests inherited from its upstream ML-KEM codebase. The Jasmin implementation runs the same kind of self-test, and additionally cross-checks its output against the C implementation, bit-for-bit, across all six builds. The ACC implementation is validated against a Python model, which subclasses `kyber-py`’s `ML_KEM`, overrides the noise sampler, and disables ciphertext compression. For each parameter set, the model generates known-answer tests (KATs) for key generation, encapsulation, and decapsulation (including implicit rejection), as well as round-trip tests. Unlike the Jasmin code, ACC’s constant-time behavior is not machine-checked; the hand-written kernels avoid secret-dependent branches by construction.

6 Benchmarks

We benchmark the C and Jasmin implementations on three Intel microarchitectures: Skylake, Coffee Lake, and Raptor Lake, with Hyper-Threading and Turbo Boost disabled. For the hybrid Raptor Lake processor, we pin the benchmark to a performance core. All code is compiled with `gcc-version-to-do`.

We compare the KAIBURR variants with FrodoKEM and HQC. We use the SHAKE-based variant of FrodoKEM as the primary comparison baseline. Both FrodoKEM-SHAKE and KAIBURR use SHAKE-based symmetric primitives. However, FrodoKEM-AES can take advantage of dedicated AES instructions on our Intel processors, which introduces an implementation-specific hardware advantage unrelated to the lattice-based KEM operations we are looking to compare. We report only FrodoKEM-640, the smallest standardized FrodoKEM parameter set, since it is already substantially more expensive than KAIBURR. The larger FrodoKEM parameter sets have even higher computational costs and do not provide a more informative performance baseline.

All three KEMs are evaluated using a common benchmark driver adapted from the implementation used in [6]. We first perform 1000 untimed warm-up iterations, followed by 10001 timed iterations. We report the median execution time and interquartile range (Q_1 – Q_3). This procedure is repeated three times, and we report the results from the run whose median is itself the median of the three runs, together with its corresponding quartiles. Each iteration verifies that the decapsulated shared secret matches the encapsulated shared secret.

Randomness comes from Linux `getrandom`. FrodoKEM and KAIBURR call it within `keypair` and `enc`, so that syscall is inside the measured region. HQC instead seeds a SHAKE-256 PRNG once, which we seed from `getrandom` before timing begins.

The benchmark results are reported separately for each processor. Results for the Intel Core i7-6700K, Intel Core i7-8700K, and Intel Core i9-13900K are given in Tables 5, 6, and 7, respectively.

We additionally benchmark the ACC implementation described in Section 5 on Pavona’s cycle-accurate RTL simulator for the accelerator. Because the simulator is deterministic for a fixed random seed, repeated runs of the same test are identical in cycles. Therefore unlike the CPU benchmarks above, we report a single exact cycle count per operation rather than a median and interquartile range. We measure each KEM operation within a single `keypair`–`encapsulate`–`decapsulate` roundtrip and record cumulative execution statistics after each operation. The individual costs are then recovered by subtraction: $\text{KeyGen} = M_1$, $\text{Enc} = M_2 - M_1$, and $\text{Dec} = M_3 - M_2$. Results are shown in Table 8.

Table 9 shows the three functions accounting for the largest share of cycles in the same roundtrip benchmark, obtained from the simulator’s per-function execution statistics. Across all three parameter sets, matrix generation (`poly_gen_matrix`) dominates, consuming 73–84% of total cycles.

`poly_gen_matrix` expands the public seed into the uniformly random matrix **A** via rejection sampling on SHAKE-128 output. Since it dominates the

Table 5. Benchmark results in cycles on an Intel Core i7-6700K. We report only the AVX2 implementations. The reference implementations are, on average, TBD percent slower for kaiburr C and TBD percent slower for kaiburr Jasmin, measured using the geometric mean. Values are reported as median (Q1–Q3).

KEM	Impl.	KeyGen	Enc	Dec
kaiburr-1792	C/asm	TBD	TBD	TBD
kaiburr-1792	Jasmin	TBD	TBD	TBD
kaiburr-4608	C/asm	TBD	TBD	TBD
kaiburr-4608	Jasmin	TBD	TBD	TBD
kaiburr-6144	C/asm	TBD	TBD	TBD
kaiburr-6144	Jasmin	TBD	TBD	TBD
FrodoKEM-640 [16]		TBD	TBD	TBD
HQC-1	[19]	TBD	TBD	TBD
HQC-3	[19]	TBD	TBD	TBD
HQC-5	[19]	TBD	TBD	TBD

ACC cycle counts, we break down its costs further. The simulator additionally reports, per function, a cycle count broken down by individual instruction. For kaiburr-6144, this instruction-level breakdown of `poly_gen_matrix` shows that rejection sampling is responsible for the vast majority of its cost. Only 464,022 of its 6,235,212 cycles (7.4%, or 6.2% of the entire benchmark) are spent stalled on `bn.wsrr` waiting for the hardware SHAKE-128 core to produce the next squeeze block. The remaining 92.6% is software cost of the rejection-sampling loop itself. 83.6% of `poly_gen_matrix`’s cycles (69.9% of the whole benchmark) are retired arithmetic instructions (`bn.and`, `bn.cmp`, and `bn.sel`) performing the mask/compare/accumulate operations of rejection sampling, and a further 8.5% (7.1% of the whole benchmark) comes from branch overhead (`bne` and `beq`) in the accept/reject loop. Table 10 summarizes this breakdown.

We repeated this instruction-level breakdown for kaiburr-1792 and kaiburr-4608 and found the same pattern. The hardware-Keccak-wait share stays flat (5.5% and 6.0% of each benchmark’s total cycles, respectively, versus 6.2% for kaiburr-6144), while the rejection-sampling arithmetic share grows with the parameter set (60.6% and 67.6%, versus 69.9% for kaiburr-6144). Table 11 gives the full comparison.

Acknowledgements

Large parts of the work were done while the first author was employed at MPI-SP.

Table 6. Benchmark results in cycles on an Intel Core i7-8700K. We report only the AVX2 implementations. The reference implementations are, on average, 540.8 percent slower for kaiburr C and 653.8 percent slower for kaiburr Jasmin, measured using the geometric mean. Values are reported as median (Q1–Q3).

KEM	Impl.	KeyGen		Enc	Dec
kaiburr-1792	C/asm	156156,(155579–157778)	159719,(159235–161416)	173015,(172106–174398)	
kaiburr-1792	Jasmin	143857,(143402–144559)	144987,(144656–145831)	150435,(149704–151098)	
kaiburr-4608	C/asm	814685,(812084–817870)	821346,(818719–824533)	848608,(845755–852000)	
kaiburr-4608	Jasmin	762767,(761322–764429)	765475,(764033–767156)	781079,(779648–782776)	
kaiburr-6144	C/asm	1367405,(1363808–1371615)	1376530,(1372953–1380951)	1411130,(1407567–1415488)	
kaiburr-6144	Jasmin	1293036,(1291067–1295424)	1295674,(1293713–1298070)	1313175,(1311174–1315640)	
FrodoKEM-640 [16]		3851932,(3851203–3859217)	4067952,(4067235–4075634)	4044333,(4043732–4053215)	
HQC-1	[19]	90229,(90070–90974)	172063,(171888–172892)	404274,(404018–404596)	
HQC-3	[19]	215176,(214956–215857)	404687,(404324–405517)	857576,(857048–858323)	
HQC-5	[19]	439786,(439458–440546)	838353,(837433–839669)	1698485,(1697851–1699766)	

References

1. Abdulrahman, A., Oberhansl, F., Pham, H.N.H., Philipoom, J., Schwabe, P., Stelzer, T., Zankl, A.: Towards ML-KEM & ML-DSA on OpenTitan. In: Blanton, M., Enck, W., Nita-Rotaru, C. (eds.) 2025 IEEE Symposium on Security and Privacy. pp. 1–19. IEEE Computer Society Press (May 2025). <https://doi.org/10.1109/SP61157.2025.00220> 5
2. Albrecht, M.R., Player, R., Scott, S.: On the concrete hardness of learning with errors. *J. Math. Cryptol.* **9**(3), 169–203 (2015), <http://www.degruyter.com/view/j/jmc.2015.9.issue-3/jmc-2015-0016/jmc-2015-0016.xml> 2
3. Almeida, J.B., Barbosa, M., Barthe, G., Blot, A., Grégoire, B., Laporte, V., Oliveira, T., Pacheco, H., Schmidt, B., Strub, P.Y.: Jasmin: High-assurance and high-speed cryptography. In: Thuraisingham, B.M., Evans, D., Malkin, T., Xu, D. (eds.) ACM CCS 2017. pp. 1807–1823. ACM Press (Oct / Nov 2017). <https://doi.org/10.1145/3133956.3134078> 5
4. Avanzi, R., Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Schwabe, P., Seiler, G., Stehlé, D.: CRYSTALS-Kyber — algorithm specifications and supporting documentation. Submission to the NIST Post-Quantum Cryptography Standardization Project (Nov 2017), <https://pq-crystals.org/kyber/data/kyber-specification.pdf>, round 1 submission, 2017-11-30 4
5. Avanzi, R., Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Schwabe, P., Seiler, G., Stehlé, D.: CRYSTALS-Kyber (version 2.0) — algorithm specifications and supporting documentation. Submission to round 2 of the NIST Post-Quantum Cryptography Standardization Project (Mar 2019), <https://pq-crystals.org/kyber/data/kyber-specification-round2.pdf>, round 2 submission, 2019-03-30 5
6. Bacelar Almeida, J., Barbosa, M., Barthe, G., Grégoire, B., Laporte, V., Léchenet, J.C., Oliveira, T., Pacheco, H., Quaresma, M., Schwabe, P., Séré, A., Strub, P.Y.: Formally verifying Kyber: Episode IV: Implementation cor-

Table 7. Benchmark results in cycles on an Intel Core i9-13900K. We report only the AVX2 implementations. The reference implementations are, on average, 498.2 percent slower for KAIBURR C and 557.8 percent slower for KAIBURR Jasmin, measured using the geometric mean. Values are reported as median (Q1–Q3).

KEM	Impl.	KeyGen		Enc	Dec
kaiburr-1792	C/asm	121724,(120872–122872)	125204,(124310–126302)	134816,(134058–136082)	
kaiburr-1792	Jasmin	141364,(140882–142142)	143058,(142620–143842)	148502,(147944–149212)	
kaiburr-4608	C/asm	648064,(645678–650674)	654366,(651728–657234)	676840,(674446–679554)	
kaiburr-4608	Jasmin	747376,(745792–749120)	749472,(747868–751250)	764490,(762874–766280)	
kaiburr-6144	C/asm	1095102,(1091646–1098922)	1097334,(1093566–1101348)	1127588,(1124030–1131712)	
kaiburr-6144	Jasmin	1272644,(1270318–1275208)	1272480,(1270246–1274906)	1285796,(1283420–1288436)	
FrodoKEM-640 [16]		3112706,(3107260–3124744)	3239826,(3233936–3249204)	3195408,(3187982–3208002)	
HQC-1	[19]	72578,(71902–73220)	139896,(138680–141026)	312692,(310798–314622)	
HQC-3	[19]	179494,(178058–180668)	332806,(329936–335788)	668334,(663984–672664)	
HQC-5	[19]	377670,(376224–380158)	705038,(700890–709262)	1357572,(1350500–1364182)	

Table 8. Benchmark results in cycles for the ACC implementation on Pavona’s cycle-accurate RTL simulator. Values are exact cycle counts.

KEM	KeyGen	Enc	Dec
kaiburr-1792	240816	239348	257474
kaiburr-4608	1453490	1424342	1467109
kaiburr-6144	2505793	2449864	2506070

- rectness. IACR Transactions on Cryptographic Hardware and Embedded Systems **2023**(3), 164–193 (Jun 2023). <https://doi.org/10.46586/tches.v2023.i3.164-193>, <https://tches.iacr.org/index.php/TCHES/article/view/10960> 5, 10, 11
- Barker, E.: Recommendation for key management: Part 1 — general. NIST Special Publication 800-57 Part 1, Revision 5, National Institute of Standards and Technology (May 2020). <https://doi.org/10.6028/NIST.SP.800-57pt1r5> 2
 - Bernstein, D.J., Chuengsatiansup, C., Lange, T., van Vredendaal, C.: NTRU prime: Reducing attack surface at low cost. In: Adams, C., Camenisch, J. (eds.) Selected Areas in Cryptography - SAC 2017 - 24th International Conference, Ottawa, ON, Canada, August 16-18, 2017, Revised Selected Papers. Lecture Notes in Computer Science, vol. 10719, pp. 235–260. Springer (2017). https://doi.org/10.1007/978-3-319-72565-9_12 1
 - Bos, J.W., Costello, C., Ducas, L., Mironov, I., Naehrig, M., Nikolaenko, V., Raghunathan, A., Stebila, D.: Frodo: Take off the ring! practical, quantum-secure key exchange from LWE. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS). pp. 1006–1018. ACM (2016). <https://doi.org/10.1145/2976749.2978425> 1, 2
 - Bos, J.W., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Schwabe, P., Seiler, G., Stehlé, D.: CRYSTALS - kyber: A CCA-secure module-

Table 9. The three functions with the largest share of total cycles in the ACC roundtrip benchmark, as a percentage of total cycles for that parameter set.

KEM	#1	#2	#3
kaiburr-1792	<code>poly_gen_matrix</code> (72.60%)	<code>basemul_acc</code> (8.08%)	<code>f4</code> (6.26%)
kaiburr-4608	<code>poly_gen_matrix</code> (80.78%)	<code>basemul_acc</code> (9.23%)	<code>f6</code> (3.96%)
kaiburr-6144	<code>poly_gen_matrix</code> (83.56%)	<code>basemul_acc</code> (9.57%)	<code>f8</code> (2.05%)

Table 10. Cycle breakdown of `poly_gen_matrix` for kaiburr-6144, obtained from the simulator’s per-instruction execution statistics. Percentages are of the whole kaiburr-6144 roundtrip benchmark (7,461,727 cycles).

Source	Cycles	% of benchmark
Rejection sampling	5,212,503	69.9%
Branch overhead	529,311	7.1%
Keccak wait	464,022	6.2%
Memory latency	29,376	0.4%

- lattice-based KEM. In: 2018 IEEE European Symposium on Security and Privacy, EuroS&P 2018, London, United Kingdom, April 24-26, 2018. pp. 353–367. IEEE (2018). <https://doi.org/10.1109/EuroSP.2018.00032> 1, 2
11. Carrier, K., Meyer-Hilfiger, C., Shen, Y., Tillich, J.: Assessing the impact of a variant of matzov’s dual attack on kyber. In: Kalai, Y.T., Kamara, S.F. (eds.) *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference*, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part I. Lecture Notes in Computer Science, vol. 16000, pp. 444–476. Springer (2025). https://doi.org/10.1007/978-3-032-01855-7_15, https://doi.org/10.1007/978-3-032-01855-7_15 2
 12. Chen, Y., Nguyen, P.Q.: Bkz 2.0: Better lattice security estimates. In: Lee, D.H., Wang, X. (eds.) *Advances in Cryptology – ASIACRYPT 2011*. pp. 1–20. Springer Berlin Heidelberg, Berlin, Heidelberg (2011) 2
 13. D’Anvers, J.P., Karmakar, A., Roy, S.S., Vercauteren, F.: Saber: Module-LWR based key exchange, CPA-secure encryption and CCA-secure KEM. In: Joux, A., Nitaj, A., Rachidi, T. (eds.) *AFRICACRYPT 18*. LNCS, vol. 10831, pp. 282–305. Springer, Cham (May 2018). https://doi.org/10.1007/978-3-319-89339-6_16 1
 14. Lucas, L., Esser, A., Etinski, S., Kirshanova, E.: Asymptotics and improvements of sieving for codes. In: Joye, M., Leander, G. (eds.) *EUROCRYPT 2024*, Part VII. LNCS, vol. 14657, pp. 151–180. Springer, Cham (May 2024). https://doi.org/10.1007/978-3-031-58754-2_6 2
 15. Ducas, L., Pulles, L.N.: Does the dual-sieve attack on learning with errors even work? In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference*, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part III. Lecture Notes in Computer Science, vol. 14083, pp. 37–69. Springer (2023). https://doi.org/10.1007/978-3-031-38548-3_2, https://doi.org/10.1007/978-3-031-38548-3_2 2

Table 11. Cycle breakdown of `poly_gen_matrix` across parameter sets, as a percentage of each variant’s total roundtrip benchmark cycles.

Source	kaiburr-1792	kaiburr-4608	kaiburr-6144
Rejection sampling	60.6%	67.6%	69.9%
Branch overhead	6.2%	6.8%	7.1%
Keccak wait	5.5%	6.0%	6.2%
Memory latency	0.3%	0.4%	0.4%

16. FrodoKEM Team and Microsoft Research: PQCrypto-LWEKE: FrodoKEM. <https://github.com/microsoft/PQCrypto-LWEKE>, gitHub repository 12, 13, 14
17. Guo, Q., Johansson, T.: Faster dual lattice attacks for solving LWE with applications to CRYSTALS. In: Tibouchi, M., Wang, H. (eds.) Advances in Cryptology - ASIACRYPT 2021 - 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6-10, 2021, Proceedings, Part IV. Lecture Notes in Computer Science, vol. 13093, pp. 33–62. Springer (2021). https://doi.org/10.1007/978-3-030-92068-5_2, https://doi.org/10.1007/978-3-030-92068-5_2 2
18. HQC Resources Team: Hqc specifications. Documentation PDF (August 2025), <http://pqc-hqc.org>, accessed: 2026-09-16 1
19. HQC Team: HQC: Hamming Quasi-Cyclic. <https://gitlab.com/pqc-hqc/hqc>, gitLab repository 12, 13, 14
20. ISO/IEC: Information technology — security techniques — encryption algorithms — part 2: Asymmetric ciphers — amendment 2. Standard ISO/IEC 18033-2:2006/Amd 2:2026, International Organization for Standardization, Geneva, Switzerland (Jun 2026), <https://www.iso.org/standard/86890.html> 1
21. MATZOV: Report on the security of LWE: Improved dual lattice attack. Report, Zenodo (Apr 2022). <https://doi.org/10.5281/zenodo.6493704> 2
22. Melo, V.D., Buchanan, W.J.: Kyfrog: A high-security lwe-based kem inspired by ml-kem (2025), <https://arxiv.org/abs/2512.06411> 2
23. National Institute of Standards and Technology: Post-quantum cryptography: Round 3 submissions. <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions> (2020), accessed: 2026-07-27 1
24. National Institute of Standards and Technology: Module-lattice-based digital signature standard. Tech. Rep. Federal Information Processing Standards Publication (FIPS) 204, U.S. Department of Commerce, Washington, D.C. (Aug 2024). <https://doi.org/10.6028/NIST.FIPS.204>, <https://doi.org/10.6028/NIST.FIPS.204> 1
25. National Institute of Standards and Technology: Module-lattice-based key-encapsulation mechanism standard. Tech. Rep. Federal Information Processing Standards Publication (FIPS) 203, U.S. Department of Commerce, Washington, D.C. (Aug 2024). <https://doi.org/10.6028/NIST.FIPS.203>, <https://doi.org/10.6028/NIST.FIPS.203> 1
26. National Institute of Standards and Technology: Stateless hash-based digital signature standard. Tech. Rep. Federal Information Processing Standards Publication (FIPS) 205, U.S. Department of Commerce, Washington, D.C. (Aug 2024). <https://doi.org/10.6028/NIST.FIPS.205>, <https://doi.org/10.6028/NIST.FIPS.205> 1

27. NIST PQC Forum: How is hqc substantially different from rlwe? NIST PQC Forum discussion (Oct 2025), <https://groups.google.com/a/list.nist.gov/g/pqc-forum/c/PBC-KPOCSF0>, accessed August 29, 2026 ²
28. PQ Code Package: mlkem-native: A unified, fast and verified ml-kem library. <https://github.com/pq-code-package/mlkem-native> (2026), gitHub repository, accessed September 17, 2026 ⁵
29. The FrodoKEM Team: FrodoKEM: Practical quantum-secure key encapsulation from generic lattices. <https://frodokem.org/#news> (2026), news section. Accessed: 2026-07-27 ¹